



Tales of the Underworld

Huțanu Andrei Leontin

Clasa a XII-a - Colegiul Național de Informatică „Tudor Vianu”
Prof. Carmen Mincă

CUPRINS

01

DESCRIERE

02

PREZENTARE GENERALĂ

03

**TEHNOLOGII
UTILIZATE**

01

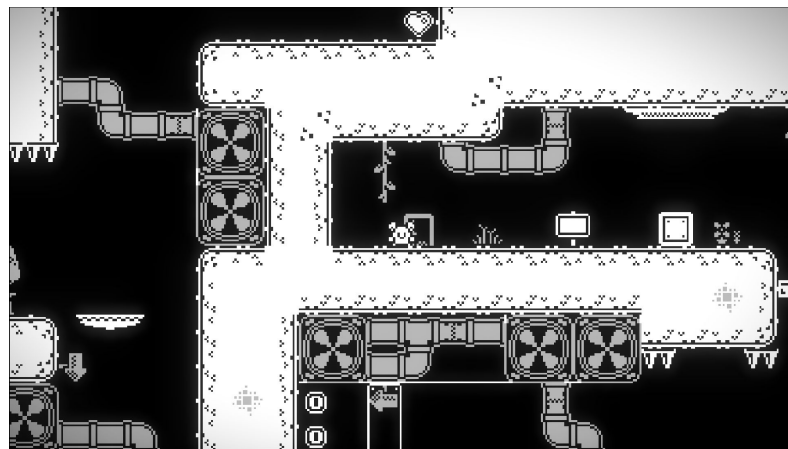
DESCRIVERE

DESCRIERE

„*Tales of the Underworld*” este un joc 2D **puzzle-platformer** minimalist, construit în jurul **manipulării gravitației**, interacțiunilor fizice și unei narațiuni scurte integrate în gameplay.

Jocul se desfășoară într-o lume alb-negru, unde claritatea vizuală este esențială, iar fiecare element din nivel are un rol funcțional. Atmosfera este susținută prin contrast puternic, efecte shader personalizate și design minimalist.

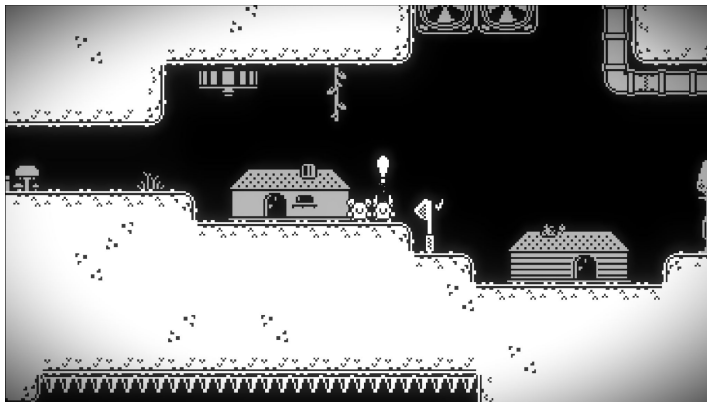
Fiecare nivel este un puzzle logic. Fiecare mecanică este o cheie.



AI – PROVOCARE ȘI OPORTUNITATE

AI APARE ÎN:

- ❖ Gestionarea stărilor (*state-based behavior*)
- ❖ Reacția la acțiunile jucătorului și deschiderea unor noi căi *în funcție de progresul* acestuia.
- ❖ Simularea unei lumi coerente



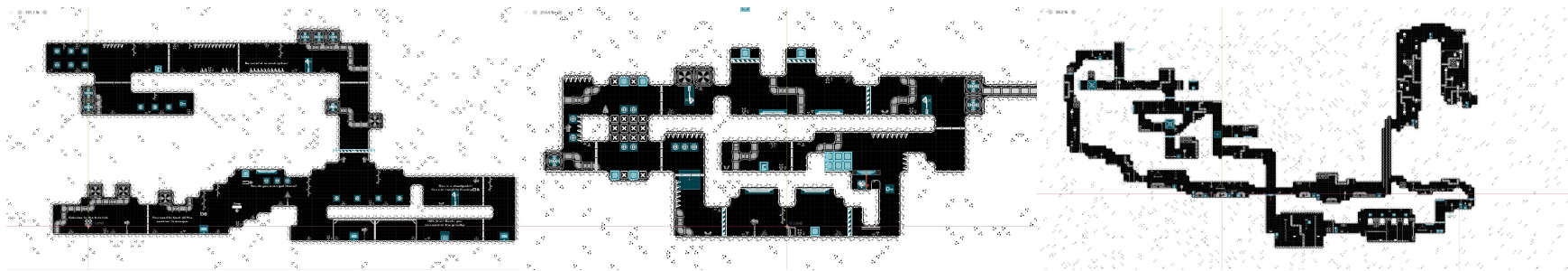
PROVOCARE

AI-ul determină modificarea dialogurilor și a reacțiilor NPC-urilor în funcție de progresul și alegerile realizate.

OPORTUNITATE

Sistemul poate produce modificări în disponibilitatea anumitor interacțiuni. Jucătorul trebuie să observe schimbările și să le interpreteze corect.

STRUCTURA JOCULUI



TUTORIAL

Destinat predării
jucătorului conceptele
de bază necesare.

NIVELURI

Fiecare nivel introduce
noi abilități și obstacole
progresiv.

OPENWORLD

Jucătorului îi este oferit un
nivel deschis, care îndeamnă
explorarea și interacțiunea cu
NPC-urile și povestea jocului.

02

PREZENTARE

IMPLEMENTARE

```
175     » » print("Key piece already obtained: " + piece_id)
176     » » return false
177
178     » key_pieces[piece_id]["obtained"] = true
179     » emit_signal("key_piece_obtained", piece_id)
180     » #print("Obtained key piece: " + key_pieces[piece_id]["name"])
181
182     » # Check if all pieces collected
183     » check_game_completion()
184
185     » return true
186
187 func has_key_piece(piece_id: String) -> bool:
188     » » if not key_pieces.has(piece_id):
189     » » » push_error("Key piece ID not found: " + piece_id)
190     » » » return false
191
192     » return key_pieces[piece_id]["obtained"]
193
194 func get_all_key_pieces() -> Array:
195     » var obtained_pieces = []
196     » for piece_id in key_pieces:
197     » » » if key_pieces[piece_id]["obtained"]:
198     » » » » obtained_pieces.append(piece_id)
199     » return obtained_pieces
200
201 # Game completion check
202 func check_game_completion() -> bool:
203     » for piece_id in key_pieces:
204     » » » if not key_pieces[piece_id]["obtained"]:
205     » » » » return false
206
207     » # All key pieces obtained, game completed!
208     » Global.GAME_COMPLETED = true
209     » print("All key pieces obtained! Game completed!")
210     » return true
211
```

Majoritatea codului legat de AI se află în două locuri:

- ❖ Sistemul de Quest-uri
- ❖ Sistemul Global

Sistemul de Quest-uri este cel care se ocupă de **verificarea validității acțiunilor jucătorului** și menținerea într-un singur loc a întregului proces care menține informații despre ultimul nivel, respectiv **interacțiunile** care au avut loc cu NPC-urile.

SISTEMUL GLOBAL

Sistemul Global înglobează majoritatea proceselor importante ale jocului și este partea centrală a acestuia. El este *punctul de legătură între toate sistemele (Quest, Scene, Tranziții, Player, UI, Culori, Muzică etc.)* și *asigură sincronizarea* informațiilor între acestea.

Scopul acestuia este de a aduce într-un loc centralizat toate *operațiile ce necesită multipli pași* pentru a asigura *reducerea posibilităților erorilor* și *simplitatea implementării sistemelor noi* pe parcursul dezvoltării.

EXAMPLE

```
90 ▾ func load_scene(wanted_scene : String, player_to_spawn_point : bool = false):
91   ▸ get_tree().get_first_node_in_group("scene_manager").change_level(wanted_scene, player_to_spawn_point)
92 ▾ func play_transition(string : String):
93   ▸ $"/root/SceneManager/TransitionManager".play_transition(string)
94 ▾ func color_manager_play_transition(string : String):
95   ▾ ▸ if invert_color:
96     ▸ ▸ $"/root/SceneManager/ColorManager".play_transition(string)
97
98 ▾ func change_music(world : int):
99   ▸ music_player.get_stream_playback().switch_to_clip(world)
```

```
417 ▾ func force_gravity_invert():
418   ▸ print("Gravity Changed")
419
420   ▸ var new_gravity = ProjectSettings.get_setting("physics/2d/default_gravity") * -1
421   ▸ ProjectSettings.set_setting("physics/2d/default_gravity", new_gravity)
422
423   ▸ var Player = get_tree().get_first_node_in_group("Player")
424   ▸ Player.is_gravity_reversed = !Player.is_gravity_reversed
425   ▸ Player.jump_power *= -1
426   ▾ ▸ if Player.is_gravity_reversed:
427     ▸ ▸ color_manager_play_transition("InvertColor_FadeIn")
428   ▾ ▸ else:
429     ▸ ▸ color_manager_play_transition("InvertColor_FadeOut")
430   ▸ Player.set_particles_gravity((-1 if Player.is_gravity_reversed else 1))
431   ▸ Player.set_scale(Vector2(Player.scale.x, abs(Player.scale.y) * (-1 if Player.is_gravity_reversed else 1)))
432   ▸ Player.has_changed_gravity = true
```

```
#GAME SETTINGS
▸ func toggle_bloom(value : bool):
▸   bloom = value
▸   world_environment.glow_enabled = value

▸ func toggle_invert_color(value : bool):
▸   ▸ if value == false:
▸     ▸ color_manager_play_transition("InvertColor_FadeOut")
▸   ▸ else:
▸     ▸ invert_color = true
▸     ▸ color_manager_play_transition("InvertColor_FadeIn")
▸   invert_color = value
▸   #get_tree().get_first_node_in_group("invert_color_canvas").visible = value
```

SISTEMUL GLOBAL

De asemenea are rolul esențial în *salvarea progresului* și al *setărilor stabilite de utilizator*, prin intermediul unor fișiere externe cu extensia „.file ” sau „.save ”. Toate informațiile necesare sunt organizate în *dicționare specifice, parsate în JSON* și amplasate în fișierele respective, loc din care pot fi mai târziu citite și jocul poate modifica setările necesare.

```
268 func load_game():
269     if not FileAccess.file_exists("res://savegame.save"):
270         return #error, file wasn't found
271
272     var save_game_file = FileAccess.open("res://savegame.save", FileAccess.READ)
273     has_save_file = true
274
275     #citeste fiecare linie
276     while save_game_file.get_position() < save_game_file.get_length():
277         var json_string = save_game_file.get_line()
278         var json = JSON.new()
279         # Creates the helper class to interact with JSON
280
281         var parse_result = json.parse(json_string)
282         #we parse and verify if it's ok
283         if not parse_result == OK:
284             print("JSON Parse Error: ", json.get_error_message(), " in ", json_string, " at line ", json.get_error_line())
285             continue
286
287         #read that line
288         var node_data = json.get_data()
289
290         #verify what it is
291         match node_data["what_is_saved"]:
292             "spawn_point":
293                 set_spawn_point(str_to_var(node_data["position"]), node_data["gravity"])
294             "currency":
295                 set_currency(node_data['amount'])
296                 CURRENT_LEVEL_CURRENCY = 0
297             "level_info":
298                 for level in node_data['levels']:
299                     levels_visited.push_back(level)
300                 for collectible in node_data['collectibles']:
301                     collectibles_got.push_back(collectible)
302             "quest":
303                 Quest.mother_spoken_to = node_data["mother_spoken_to"]
304                 Quest.husband_found = node_data["husband_found"]
305                 Quest.money_found_1 = node_data["money_found_1"]
306                 Quest.child_found = node_data["child_found"]
307                 Quest.selfish_spoken_to = node_data["selfish_spoken_to"]
308                 Quest.money_found_2 = node_data["money_found_2"]
309
310         print("Game Loaded")
```

03. TEHNOLOGII

PHOTOPEA

Program online **gratuit** pentru editare foto.

Utilizat pentru:

- ❖ Sprite-uri
- ❖ UI minimalist
- ❖ Elemente grafice pentru niveluri

GODOT ENGINE 4

- ❖ Randare **Vulkan** eficientă.
- ❖ Sistem de fizică 2D integrat, modificat pentru utilizarea specifică a jocului (**gravitație inversată/orizontală**).
- ❖ **GScript**.
- ❖ Sistem de scene modular și sistem de semnale publice.
- ❖ **Autoload scripturi** prin intermediul **singletonurilor**.
- ❖ Suport pentru shadere personalizate.
- ❖ **Sistem Mixer** pentru sunete pe canale diferite.

Funcționalități utilizate:

- ❖ CharacterBody2D, modificat.
- ❖ RayCast2D pentru detectarea coliziunilor.
- ❖ Collision Layers & Masks pentru filtrarea straturilor de coliziune.
- ❖ CanvasLayer pentru post-procesare.
- ❖ AudioBus pentru muzică și efecte sonore.



ACCESIBILITATE:

Jocul include:

- ❖ Ajustare luminozitate & contrast.
- ❖ Dezactivare invertire culori.
- ❖ Toggle pentru bloom.
- ❖ Design alb-negru pentru lizibilitate crescută.

RESURSE:

- ❖ [Kenney.nl](https://kenney.nl) - Grafica de bază.
- ❖ [Pixabay.com](https://pixabay.com) - Efecte sonore.
- ❖ sfxr.me - Efecte sonore 8-bit.
- ❖ Muzica proprie.

Made By
Hutanu Andrei

WITH LOVE 